An Empirical-Based Security Benchmarking Framework for Container Confinement

by

Huzaifa Patel

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs
in partial fulfillment of the requirements for the degree of

Master
in
Computer Science

Carleton University
Ottawa, Ontario

© 2025 Huzaifa Patel

Contents

1 Background **4**
- 1.0.1 Type 1 Hypervisor . . . 5
- 1.0.2 Type 2 Hypervisor . . . 5
- 1.0.3 Problems With Type 1 & Type 2 Hypervisor Classifications . 5
- 1.0.4 Native Hypervisor . . . 6
- 1.0.5 Emulation Hypervisor . . . 6
- 1.0.6 Why do we have run Docker with sudo? . . . 6
- 1.0.7 YAMA . . . 7
- 1.0.8 Kernel Implementation . . . 9
- 1.0.9 Enforcement of `ptrace_scope` . . . 10
- 1.0.10 `ptrace_scope` Settings . . . 11

2 Design **12**
- 2.1 Reproducibility and Deterministic Execution . . . 12
- 2.2 Runtime Injection via OverlayFS . . . 13
- 2.3 Preventing Environmental Drift . . . 14
  - 2.3.1 Determinism vs. Reproducibility . . . 15
  - 2.3.2 Ephemeral VM Lifecycle as a Defense . . . 16

3 Evaluation **17**
- 3.0.1 Purpose . . . 17
- 3.0.2 Requirements / Goals: . . . 17
- 3.0.3 Methodology / How we will do that: . . . 19
- 3.0.4 What Did You Find Out? . . . 20
- 3.1 Reproducibility Across Time with Version Pinning . . . 22
  - 3.1.1 Experimental Setup . . . 23
  - 3.1.2 AppArmor Enforcement Trick Logic . . . 24
  - 3.1.3 Initial Execution (T_0) . . . 25
  - 3.1.4 Re-run Execution (T_1) . . . 26
  - 3.1.5 Reproducibility Metric . . . 26
  - 3.1.6 Analysis and Observations . . . 27
  - 3.1.7 Conclusion . . . 27
- 3.2 Host Isolation Validation . . . 27
  - 3.2.1 Threat Model . . . 28
  - 3.2.2 Malicious Trick Description . . . 29

3.2.3 Execution Inside the VM 30
3.2.4 Implications of Shared Folders 30
3.2.5 Baseline Comparison: Without VM Isolation 31
3.2.6 Consideration of VM Escape Attacks 32
3.2.7 Scope and Limitations. 34
3.2.8 Conclusion 34
3.3 Extensibility Evaluation 35
3.4 Version Variability Support 38
3.5 Performance Benchmarking 42
3.5.1 Microbenchmark 43
3.6 Batch Trick Execution 48

4 Case Studies **49**
4.0.1 Case Study 1: Evaluating the accuracy of CVE-2024-21626 . . 49
4.0.2 Case Study 2 51
4.0.3 Case Study 3 - Evaluating Docker Privileged Port Forwarding Restrictions 54
4.0.4 Case Study 4 - Evaluating ptrace Enforcement Across Linux Distributions 56

Nomenclature

VM virtual machine. 3

Chapter 1

Background

The existance of cloud computing can be attributed to virtualization, which is a technology that makes it possible for multiple different operating systems (OSs) to run concurrently, and in an isolated environment on the same hardware. How does virtualization achieve this? It makes use of a machines hardware to support the software that creates and manages virtual machines (VMs). A virtual machine (VM) is a virtual environment that provides the functionality of a physical computer by using its own virtual central processing unit (VCPU), memory, network interface, and storage. The software that creates and manages VMs is formally called a hypervisor or virtual machine monitor (VMM). The virtualization marketplace is comprised of four notable hypervisors, which are: (1) VMWare, (2) Xen, (3) Kernel-based Virtual Machine (KVM), and (4) Hyper-V. The operating system running a hypervisor is called the host OS, while the VM that uses the hypervisor is called the guest OS. To maintain consistency within this proposal, we will use the word "hypervisor" when referring to virtualization software. Hypervisors can be divided into two types, depending on where they are located on the machine: (1) type 1 and (2) type 2.

1.0.1 Type 1 Hypervisor

Type 1 hypervisors run directly on physical hardware to create, control, and manage VMs, and do not require support from the host OS. Type 1 hypervisors are also called native or bare-metal hypervisors. The first hypervisors, which IBM developed in the 1960s were type 1 hypervisors [?]. Examples of type 1 hypervisors include, but are not limited to Xen, KVM, VMware ESX, and Microsoft Hyper-V [?].

1.0.2 Type 2 Hypervisor

Type 2 hypervisors (also called hosted hypervisors) are installed on the OS (Windows, Linux, MacOS), similar to how computer programs are installed onto an OS. In other words, a type 2 hypervisor runs as a process on the host OS. Type 2 hypervisors abstract guest OSs from the host OS by introducing a third software layer above the hardware, as shown in figure 2.1. Examples of type 2 hypervisors include but are not limited to VMware Workstation, VirtualBox, and QEMU [?].

1.0.3 Problems With Type 1 & Type 2 Hypervisor Classifications

Although the definitions of type 1 and type 2 hypervisors are widely accepted, there are gray areas where the distinction between the two remain unclear. For instance, according to its creator RedHat, KVM is implemented and deployed using two Linux kernel modules that effectively convert the host Linux OS into a type-1 hypervisor [?]. At the same time, KVM can be categorized as a type 2 hypervisor because KVM VM's are standard Linux processes that are competing with other Linux processes for CPU time given by the Linux kernel's native CPU scheduler [?]. Due to disagreements and vagueness in the definitions of type 1 and type 2 hypervisors, two new classifications were defined: (1) native hypervisors and (2) emulation hypervisors [?].

1.0.4 Native Hypervisor

Native hypervisors are hypervisors that push VM guest instructions directly to the physical machines CPU, by using virtualization extensions like Intel VT-x. [**?**]. Examples of Native hypervisors include but are not limited to Xen, KVM, VMware ESX, and Microsoft HyperV.

1.0.5 Emulation Hypervisor

Emulation hypervisors are hypervisors that emulate every VM guest instruction using software virtualization [**?**]. Emulated guest instructions are very easy to trace because all guest VM instructions are trapped to the hypervisor. Examples of emulation hypervisors include but are not limited to QEMU, Bochs, early versions of VMware-Workstation, and VirtualBox [**?**].

1.0.6 Why do we have run Docker with sudo?

By default, Docker's daemon (dockerd) runs as root, and access to it is controlled through the Unix socket:

Copy Edit /var/run/docker.sock This socket is owned by root and typically part of the docker group. If your user isn't in the docker group, you'll need sudo to interact with it.

Why Docker Needs Elevated Privileges: Container Isolation Requires Kernel Features:

Managing cgroups, namespaces, mounting filesystems, setting up virtual networks—all require elevated permissions.

By Default, Docker Protects Access to Its Control API:

docker.sock is a privileged control interface that allows full system control (you can effectively become root via it).

1.0.7 YAMA

Yama is a Linux Security Module (LSM) that adds additional restrictions to the ptrace system call, which is commonly used for debugging and process tracing. While ptrace is a powerful tool for legitimate uses like debuggers and profilers, it can also be abused by malicious processes to interfere with or spy on other running processes. Yama was introduced to mitigate these risks by controlling and limiting which processes can use ptrace on others, providing a simple but effective hardening mechanism against certain types of attacks.

The core of Yama's functionality revolves around the /proc/sys/kernel/yama/ptrace_scope setting, which controls how restrictive the kernel is regarding ptrace access. The available levels range from 0 to 3, each enforcing progressively stricter controls. With ptrace_scope=0, the system behaves traditionally, allowing any process to ptrace another process owned by the same user if the target is marked as dumpable. At 1, only parent processes can trace their children, limiting the risk of arbitrary tracing. Level 2 requires a process to have CAP_SYS_PTRACE capabilities (effectively root-level permissions) to trace any process, and level 3 completely disables ptrace system-wide, locking down the system from process tracing even by privileged users.

Yama is especially valuable in multi-user systems or environments where processes run under the same user account but should remain isolated from one another, such as in containerized or sandboxed environments. While it is a relatively lightweight security module compared to more comprehensive systems like SELinux or AppArmor, it adds a meaningful layer of defense against local privilege escalation attacks. However, it's important to balance its restrictions with the need for debugging and profiling tools that rely on ptrace. In production environments focused on security, higher ptrace_scope values (such as 2 or 3) may be appropriate, while development systems typically keep it at 0 or 1 to allow standard debugging workflows.

Yama does not enforce its ‘ptrace_scope‘ restrictions when attempting to trace host processes from within a container because Yama's enforcement is limited to the current process visibility defined by PID namespaces. In most container environments, the containerized process runs in an isolated PID namespace and cannot see or interact with host processes unless explicitly configured using options like ‘–pid=host‘. Even when a container is started with ‘–pid=host‘, allowing it to see host processes, Yama primarily governs interactions within the same namespace context and is not designed to enforce security policies across different namespaces.

Additionally, for Yama to restrict tracing of host processes, the containerized process would not only need visibility of host PIDs but also require sufficient privileges, such as ‘CAP_SYS_PTRACE‘, in the host's user namespace. In practice, other security mechanisms like AppArmor, SELinux, and seccomp often prevent such privilege escalation before Yama's policies even apply. As a result, Yama's effectiveness is primarily confined to preventing ptrace interactions between processes that are already within the same visibility scope, rather than serving as a cross-namespace isolation mechanism. This makes Yama a useful tool for hardening processes within the same environment but not a comprehensive defense against container-to-host process tracing attempts.

The behavior of Yama is controlled by the `/proc/sys/kernel/yama/ptrace_scope` setting, which determines the level of restriction imposed on `ptrace` operations. The possible values are summarized below:

When Yama's ptrace_scope is set to 2, processes cannot freely attach to other unrelated processes, even if they have administrative capabilities or are running as root. In this mode, the ability to trace or debug a process must be explicitly granted by the target process itself or configured through system-level controls.

This permission can be granted using the prctl() system call with the PR_SET_PTRACER option, allowing a specific process to act as a tracer. Alternatively, the system ad-

Table 1.1: Yama `ptrace_scope` Settings

Value	Behavior
0	No additional restrictions. Processes can trace any other process they have permissions for.
1	Restricted to child processes only. This is the default setting in many distributions.
2	Restricted unless explicitly allowed via `/proc/[pid]/uid_map` or using `prctl()`.
3	No `ptrace` allowed at all, even for processes with `CAP_SYS_PTRACE`. This is the most restrictive setting.

ministrator can modify the global ptrace policy by lowering the ptrace_scope value in /proc/sys/kernel/yama/ptrace_scope. Without such explicit authorization, ptrace attempts will consistently fail, regardless of user privileges or capabilities. This design enforces stronger security boundaries by preventing unauthorized inspection or manipulation of processes, even by privileged users, unless the target process explicitly consents to being traced.

Yama is a Linux Security Module (LSM) that enforces additional restrictions on the `ptrace` system call and process debugging mechanisms. Its primary purpose is to reduce the attack surface for exploits that rely on process introspection and manipulation. By limiting the ability of processes to trace or interact with others, Yama enhances system security and makes privilege escalation attacks more difficult. This is particularly valuable in containerized environments and multi-user systems where process isolation is critical.

1.0.8 Kernel Implementation

Yama is implemented in the Linux kernel source file `security/yama/yama_lsm.c`. It integrates with the kernel's LSM framework by registering specific hooks that intercept and evaluate `ptrace` operations. During initialization, Yama registers its hooks using the following function:

```
void __init yama_add_hooks(void)
{
    pr_info("Yama: becoming mindful.\n");
    security_add_hooks(yama_hooks, ARRAY_SIZE(yama_hooks), "yama");
    yama_init_sysctl();
}
```

Here, `security_add_hooks` registers an array of hook functions defined in `yama_hooks`. These hooks include:

- `ptrace_access_check`
- `ptrace_traceme`
- `task_prctl`
- `task_free`

Each hook is associated with a specific kernel operation, allowing Yama to enforce its security policies at critical points.

1.0.9 Enforcement of `ptrace_scope`

The core of Yama's enforcement logic is implemented in the `yama_ptrace_access_check` function:

```
static int yama_ptrace_access_check(struct task_struct *child,
         unsigned int mode){
}
```

This function determines whether a `ptrace` operation should be permitted based on the current `ptrace_scope` value and the relationship between the tracer and tracee processes.

1.0.10 `ptrace_scope` Settings

Yama provides four levels of `ptrace` restrictions, configurable via `/proc/sys/kernel/yama/ptrace_`

These settings are summarized in Table 1.1.

Chapter 2

Design

2.1 Reproducibility and Deterministic Execution

A central design goal of Houdini is to ensure that container security tests are reproducible and deterministic. Reproducibility means that when a test is run multiple times under the same conditions, it should produce the same result. Determinism ensures that given the same inputs, including the kernel version, filesystem state, runtime binaries, and test logic, the system will always produce the same outputs, without being influenced by system randomness, entropy, or external changes.

This level of control is especially important in security testing, where consistency is critical. Without strict version control, even small system updates can lead to changes in container behavior. For example, an operating system update might alter how system calls are handled, or a new version of Docker or runc might introduce subtle changes in default behavior. These differences can cause a trick to unexpectedly pass or fail, even though the code itself has not changed.

Such inconsistencies lead to misleading test results. A trick might appear to succeed because of quirks in the environment rather than an actual vulnerability (a false positive), or it might fail due to an unrelated system update that blocks the exploit (a

false negative). In both cases, the conclusions drawn from the test cannot be trusted. This is especially problematic in workflows like coordinated vulnerability disclosure, where researchers and vendors must independently verify the exploit under identical conditions, or in compliance audits that require consistent, repeatable testing.

To eliminate these risks, Houdini is designed with a tightly controlled execution environment. All critical components like Docker, runc, and the Linux kernel are version-pinned. Their binaries are precompiled and injected into the VM using an overlay filesystem, bypassing reliance on external package managers or distribution-level dependencies. This ensures that the exact versions of all runtime components are known and fixed.

Additionally, Houdini builds its guest environment using Buildroot, a minimalist embedded Linux build system. This allows precise control over every component included in the VM. The result is a clean, self-contained execution environment that avoids the sources of nondeterminism commonly found in general-purpose distributions, such as background services, automatic updates, or untracked dependencies.

In summary, Houdini's design guarantees that a trick executed today will behave the same way when executed days, weeks, or months later, provided that the same version-pinned environment is used. This reliability is critical for evaluating security vulnerabilities, verifying patches, performing longitudinal studies, and conducting reproducible research. By enforcing a deterministic testbed, Houdini isolates the cause of behavioral differences to the trick logic or configuration itself—never to hidden changes in the surrounding infrastructure.

2.2 Runtime Injection via OverlayFS

Rather than relying on Buildroot's package infrastructure to download and install Docker and runc inside the VM, we used an overlayfs directory to inject precompiled

binaries of both components directly into the root filesystem. This approach provides full control over the runtime environment and ensures that only explicitly specified versions are used during testing. We chose not to use Buildroot packages because doing so introduced limitations: several historical versions of Docker and runc were either unavailable or failed to build consistently within Buildroot's package ecosystem. By manually sourcing and verifying the binaries, we were able to include a broader range of versions and maintain compatibility across all tests.

2.3 Preventing Environmental Drift

Environmental drift refers to unintentional changes in the system that can affect the outcome of container-based tests. These changes may include automatic updates to the operating system, differences in kernel versions, updated system libraries, or changes in container runtime behavior. Even when a test script remains the same, such background variation can cause container behavior to differ between runs—leading to inconsistent or misleading results.

This is a critical concern for security testing, where consistency is essential. Without environmental stability, researchers may encounter false positives, where a trick appears to succeed because of an incidental change, or false negatives, where a real vulnerability is masked by a system-level patch or configuration shift. In either case, the test results become untrustworthy, and any security claims based on those results may be invalid.

To address this, Houdini is designed to eliminate sources of environmental drift by creating a fully version-controlled, deterministic testing environment. All critical components that influence container behavior—including the Linux kernel, Docker engine, and runc runtime—are manually sourced, precompiled, and injected into the root filesystem using an overlay filesystem. This ensures that Houdini uses only ex-

plicitly specified versions of each component, independent of host updates or package manager behavior.

This design choice was necessary because many historical versions of Docker and runc are either unavailable through Buildroot's package system or fail to build due to missing dependencies or upstream changes. By bypassing the package infrastructure and directly injecting verified binaries, Houdini achieves a consistent and reproducible runtime environment across all test sessions.

Additionally, Houdini uses Buildroot to generate a minimal, static guest operating system that excludes background services, update agents, or untracked dependencies that could introduce nondeterminism. The virtual machine is launched without networking to prevent environmental noise from DNS, package mirrors, or remote services. Only a single, tightly scoped communication channel is exposed to the host through a QEMU user-mode forwarded port used for reporting results.

Together, these design decisions ensure that Houdini provides a frozen, deterministic environment where test outcomes are reproducible over time. This stability is especially important for longitudinal analysis, vulnerability tracking, patch verification, and reproducible research—where changes in the results must reflect actual differences in the tested logic, not unintended changes in the surrounding system.

2.3.1 Determinism vs. Reproducibility

While determinism ensures that a given input will yield the same output under identical conditions, reproducibility additionally requires that the environment itself remain stable over time. Houdini addresses both concerns: determinism is enforced by controlling the trick's logic, while reproducibility is achieved through version pinning, overlay-based injection, and hash verification.

2.3.2 Ephemeral VM Lifecycle as a Defense

Houdini's architecture launches a clean VM for each test. Even if a trick succeeds in compromising the guest kernel or container runtime, the damage is limited to that ephemeral instance. The VM is destroyed upon test completion, and its state does not persist. This provides a form of temporal isolation that complements spatial isolation, making persistence and lateral movement unfeasible for attackers.

Chapter 3

Evaluation

3.0.1 Purpose

The purpose of this evaluation is to empirically assess the effectiveness, reliability, and practical utility of Houdini as a framework for detecting security issues, misconfigurations, and enforcement inconsistencies in containerized environments.

3.0.2 Requirements / Goals:

- **Reproducibility:** Houdini should produce consistent and repeatable results when run across different machines and environments. Identical configurations and test inputs should yield the same outcomes.
- **Scalability:** Houdini should support scalable execution, allowing users to easily configure, queue, and execute large batches of tests/tricks across multiple container configurations or versions. The framework should be designed to accept input declaratively and automate the full evaluation without requiring manual intervention.
- **Performance:** Houdini should execute tests efficiently, minimizing runtime overhead while maintaining accuracy. The framework's performance should

allow it to be integrated into larger workflows, such as continuous integration pipelines, without introducing significant delays.

- **Security and Host Isolation:** Houdini must isolate tests from the host system to ensure safety during the execution of potentially dangerous tricks. By running tests inside a virtual machine, Houdini should prevent container escapes or exploits from impacting the host. It should maintain a secure boundary between the test environment and the underlying system.

- **Accuracy of Detection** Houdini should correctly distinguish between successful and failed security isolation. Test results should reflect the actual runtime behavior of the container under evaluation without false positives or negatives.

- **Flexibility Across Configurations** The framework should support a wide range of container settings, including different Linux capabilities, namespaces, AppArmor profiles, seccomp filters, and Docker flags. It should also accommodate various versions of Docker, runc, crun, and the Linux kernel.

- **Transparency and Debuggability** Test runs should be transparent and easy to interpret. Logs, intermediate outputs, and error messages should be informative enough to help users understand why a trick passed or failed.

- **Version Pinning**

- **Detect Configuration-Behavior Inconsistencies:** Evaluate whether containers behave as expected under different security configurations by empirically validating real-world runtime behavior. This helps identify mismatches between intended policy and actual enforcement.

- **Assess Cross-Version Consistency and Enforcement Drift:** Determine whether the same container configuration yields consistent behavior across dif-

ferent versions of Docker, container runtimes (e.g., runc, crun), and Linux kernels.

3.0.3 Methodology / How we will do that:

To evaluate the effectiveness and design properties of the Houdini framework, we conduct a series of controlled experiments targeting key aspects of its architecture and operation. Each experiment is designed to test a specific goal of the framework, including reproducibility, scalability, performance, and security isolation.

We structure our methodology as follows:

1. **Reproducibility Across Machines**

   Run the same trick and configuration on two different machines (e.g., different CPU architectures or hardware specs) and verify that the output is identical.

2. **Reproducibility Across Time with Version Pinning ✓**

   Re-run a pinned Docker, kernel, and runc configuration at a later time to ensure consistent results, demonstrating Houdini's ability to reproduce experiments reliably.

3. **Scalability via Bulk Execution ✓**

   Execute a large batch of tricks (e.g., 32) across multiple configurations in one session. Measure runtime, memory usage, and failure rates to assess Houdini's batch execution capabilities.

4. **Performance Benchmarking ✓**

   Measure the end-to-end time it takes Houdini to execute a single trick, including VM boot, container launch, and result collection. Compare this to a manually executed baseline outside Houdini.

5. **Host Isolation Validation ✓**

   Run a deliberately malicious trick (e.g., attempting to write to host directories) and verify that the host remains secure, demonstrating that Houdini's use of VMs effectively isolates test environments.

6. **Fault Tolerance and Crash Recovery**

   Introduce failing or crashing tricks (e.g., segmentation faults, infinite loops) and confirm that Houdini logs the failure and continues running subsequent tests without interruption.

7. **Version Variability Support ✓**

   Create a test matrix across different combinations of Docker, runc, and kernel versions. Confirm that Houdini successfully builds and runs tricks across all variations.

8. **Extensibility Test ✓**

   Develop a new custom trick (e.g., testing for a capability or syscall) and integrate it into Houdini. Measure the effort and steps needed to add and run it without modifying the core logic.

9. **CI Integration Test**

   Integrate Houdini into a CI/CD system such as GitHub Actions or GitLab CI. Trigger it on container configuration changes and assess its stability and reporting in an automated pipeline.

3.0.4 What Did You Find Out?

TODO when I actually run the experiments, but for the last two goals I have some results:

- The goal **"Detect Configuration-Behavior Inconsistencies"** is directly

supported by **Case Studies 2, 3, and 4**, where the same configurations sometimes led to unexpected or insecure behavior (e.g., host interaction without capabilities, port binding despite restrictions, and ptrace behavior differing by scope).

- The goal **"Assess Cross-Version Consistency and Enforcement Drift"** is supported by **Case Studies 1, 3, and 4**, where the same trick or config yielded different results across versions of Docker, runc, and Linux distributions.

- **Case Study 1 (CVE-21626):** Discovered that older versions of `runc` (e.g., rc1–rc4), not listed in the official CVE disclosure, were also vulnerable. This expanded the known scope of the vulnerability and revealed version-scoping inconsistencies.

- **Case Study 2 (Host Process Manipulation):** Identified the minimum set of privileges required for a container to successfully interact with host processes. Demonstrated that certain combinations of namespace sharing and UID matching can bypass expected isolation controls, even with reduced capabilities.

- **Case Study 3 (Docker Privileged Port Binding):** Found that containers could bind to privileged ports under newer Docker versions despite appearing unprivileged. This revealed misconfigurations and inconsistencies between intended policy and actual runtime behavior.

- **Case Study 4 (ptrace Enforcement Across Linux Distributions):** Empirically validated that ptrace enforcement behavior varies across Linux distributions due to differences in kernel versions and default Yama settings. Exposed enforcement drift[1] and platform-dependent inconsistencies in confinement guarantees.

[1]Drift refers to changes in security behavior over time due to image updates, runtime automation, or infrastructure modifications.

3.1 Reproducibility Across Time with Version Pinning

This experiment evaluates Houdini's ability to reliably reproduce test results over time using a version-pinned container environment. The purpose of this experiment is to determine whether Houdini can serve as a reliable and deterministic testing framework when executed repeatedly over time under controlled, version-pinned conditions. Version pinning refers to explicitly locking the versions of software components, such as Docker, runc, and the Linux kernel, to prevent variation due to automatic updates, rebuilds, or environmental drift. Even slight changes in runtime binaries, system dependencies, or build environments can introduce behavioral drift that invalidates prior findings and undermines test integrity.

To test reproducibility across time, we define two time points: T_0, the initial execution, and T_1, a later re-execution under identical conditions. If Houdini produces the same outputs at both time points, reproducibility is confirmed. We hypothesize that if the trick stays the same and the software environment doesn't change, Houdini will give the same test results every time. This would make it a reliable tool for running long-term container security tests.

Some tricks may always produce the same result, even if environmental drift occurs. In those cases, the result looks consistent, but it does not mean the environment stayed the same. An example of a trick that always produces the same result even when Docker, `runc`, or the kernel changes is a simple check for the presence of a file inside the container. For instance, the following script verifies whether a specific file exists:

Listing 3.1: Trick that is invariant to environment changes

```
1 #!/bin/sh
2 if [ -f /run/trigger_file ]; then
```

```
    echo "Trick passed"
else
    echo "Trick failed"
fi
```

In this case, the output is solely a function of the script logic and the presence or absence of a file. It depends entirely on the internal state of the container's filesystem and not on any external behavior of Docker, `runc`, or the kernel. There are no side channels, timing variations, or kernel-level mechanisms that affect the result. This deterministic behavior is a direct consequence of the scripting language itself and the semantic guarantees of shell evaluation. Because the script is self-contained, the result remains the same regardless of surrounding changes—unless those changes directly affect the file being checked. This highlights the importance of distinguishing between true environmental consistency and tricks that are trivially reproducible due to their independence from external state.

3.1.1 Experimental Setup

Table 3.1: Virtual Machine Experimental Configuration

Parameter	**Value**
Virtualization Platform	QEMU
vCPUs	4
RAM	12 GB
Root Filesystem	ext2 (minimal)
Kernel Version	Linux 6.0 (custom build)

Table 3.2: Version-Pinned Components

Component	**Pinned Version**
Docker	27.0.3
runc	1.1.13
Linux Kernel	6.0

To ensure binary-level consistency between runs T_0 and T_1, we calculated the SHA256 hashes of each component and stored them in a version manifest file. These hashes act as a cryptographic fingerprint to confirm that the binaries remain unchanged throughout the experiment. These values were re-verified at the time of re-execution using the `sha256sum` utility to confirm that no environmental drift had occurred.

Table 3.3: Reproducibility Verification Across Time

Component	**SHA256 @ T0**	**SHA256 @ T1**
Docker	`898c...5e3`	`898c...5e3`
runc	`4799...0a1`	`4799...0a1`
Kernel	`3b03...1c7`	`3b03...1c7`

3.1.2 AppArmor Enforcement Trick Logic

The AppArmor detection trick is used to check if the container is running with AppArmor security enabled, specifically under the ‘docker-default‘ profile. It’s a simple and effective way to see if Linux’s AppArmor is active inside the container. The trick works by reading the file ‘/proc/1/attr/current‘, which shows the AppArmor profile applied to the container’s main process. If the profile includes ‘docker-default‘, the trick passes, meaning AppArmor is working correctly. If the profile is missing or different, the trick fails, which could mean that AppArmor is turned off, not supported, or misconfigured. This trick helps identify whether the container’s security setup

has changed across different Docker versions, kernel updates, or system settings. It's useful for spotting security drift that might affect how containers are confined.

The `apparmor_check.sh` script is placed in `/usr/bin/` and included via the overlay. Its purpose is to detect whether Docker applies AppArmor confinement, which varies by Docker version. The script attempts to write to /proc/sys/kernel/randomize_va_space, a location typically blocked by the default AppArmor profile.

Listing 3.2: AppArmor Enforcement Trick Script

```
#!/bin/sh

echo 1 > /proc/sys/kernel/randomize_va_space 2>/dev/null

if [ $? -eq 0 ]; then
    echo "Trick failed: AppArmor not enforced"
else
    echo "Trick passed: AppArmor enforcement detected"
fi
```

This trick is sensitive to Docker version behavior. Older Docker versions may not apply AppArmor profiles by default, resulting in successful writes and a failed trick. In contrast, modern versions typically enforce the `docker-default` profile, causing the trick to pass.

3.1.3 Initial Execution (T_0)

At time T_0, the environment was constructed using the pinned versions of Docker, runc, and the Linux kernel. The container was run with the AppArmor trick script. All binary hashes were verified against the manifest, confirming that the test environment matched the intended configuration exactly.

Listing 3.3: Running the AppArmor Trick in a Container

```
docker run --rm myimage /apparmor_check.sh
```

Result: Trick passed

3.1.4 Re-run Execution (T_1)

At time T_1, after a delay, the Buildroot image was rebuilt using the same configuration and the same overlay directory. This ensured that all injected binaries were identical to those used at T_0. All component hashes matched the entries in `version_manifest.txt`, confirming no binary drift had occurred. The container was again executed using the same AppArmor trick.

Listing 3.4: Running the AppArmor Trick in a Container (T_1)

```
docker run --rm myimage /apparmor_check.sh
```

Result: Trick passed

3.1.5 Reproducibility Metric

We define a basic reproducibility metric R as the percentage of test cases for which the result at time T_0 matched the result at time T_1:

$$R = \frac{\#\text{ of consistent test results}}{\text{total test cases}} \times 100\%$$

For this experiment:

$$R = \frac{2}{2} \times 100\% = 100\%$$

This confirms full reproducibility under a pinned environment.

3.1.6 Analysis and Observations

This experiment illustrates that when the container runtime stack is fully controlled through version pinning via overlay injection, Houdini can achieve bit-for-bit reproducibility. These controls help make sure the test isn't affected by updates, build changes, or system differences over time. The trick attempts a privileged action that is only blocked if security policies are enforced. So, if the result changes, it could reflect a meaningful shift in runtime behavior or security policy enforcement. That said, if Docker's AppArmor behavior never changes across versions, this trick would produce consistent results, highlighting that not all security changes are observable through every trick.

3.1.7 Conclusion

This experiment shows that Houdini can consistently reproduce results over time, making it a reliable tool for long-term and repeatable security testing. By locking in the versions of all key components, checking file integrity, and using a security-sensitive trick, Houdini avoids false consistency and detects real changes when they occur. While this test used an AppArmor enforcement check, the same approach can be used for more complex security tests like checking for exploits, verifying patches, or running compliance checks where stable and repeatable results are essential.

3.2 Host Isolation Validation

This experiment is designed as an informal security evaluation rather than a rigorous formal verification. Its goal is not to prove—beyond all possible doubt—that VM escape vulnerabilities are absent. Instead, the aim is to provide practical evidence that, under typical conditions, Houdini's architecture does not expose any clear or unintended paths through which a containerized workload could influence or compromise

the physical host.

To assess this, we use a controlled test: a container attempts to write to a file path that would typically point to a host-mounted directory, if such sharing were accidentally enabled. This simulated path acts as a canary, alerting us to any misconfigurations or unexpected leakages, such as shared folders or improperly scoped namespaces, that could break the intended isolation. If the container is able to write to this location, it would indicate a breach in the containment model.

While this approach cannot detect deeper issues such as hypervisor vulnerabilities, low-level device exploits, or side-channel attacks, it is well-suited for detecting more common and operational misconfigurations. In real-world scenarios, these kinds of setup errors are often the cause of unintended host exposure. Thus, this method offers a practical way to confirm that Houdini's default setup maintains strong separation between the host and guest environments.

A full formal security analysis—including techniques like fuzzing, symbolic execution, or hypervisor introspection—would provide deeper coverage but is out of scope for this work.

3.2.1 Threat Model

The threat model for this evaluation assumes that the attacker has full control over arbitrary code executed inside a container running within Houdini's VM. This reflects a realistic scenario where untrusted or potentially malicious container workloads are tested to evaluate security boundaries. The attacker may attempt to exploit kernel vulnerabilities, access unintended resources, or perform file system operations that target the host. Typical actions include attempting to write to host-mounted directories, reading sensitive data, or interacting with other host processes.

However, the attacker's capabilities are strictly confined to the VM. They have no access to the physical host machine, nor are they granted elevated privileges beyond

what is available inside the containerized guest. The container may share namespaces with the host, and the kernel it interacts with is the one booted inside the VM, not the host kernel.

However, in Houdini's design, these namespace-sharing configurations are restricted to the context of the VM only. That is, even if a container is launched with `--pid=host`, it shares the PID namespace only with the VM, not the physical host. Likewise, shared network or IPC namespaces remain confined to the QEMU instance. This ensures that while Houdini supports evaluation of shared-namespace behavior, any unintended access or compromise remains sandboxed inside the VM. The attacker cannot affect the physical host's kernel or global namespaces, and cannot persist beyond the lifecycle of the VM.

The goal of this threat model is to validate that Houdini can safely execute such container scenarios without compromising the integrity of the physical host. Even if the attacker succeeds in exploiting the container or VM environment, the outer boundary provided by QEMU must prevent any direct impact on the host.

3.2.2 Malicious Trick Description

We designed a trick to evaluate whether a container running inside Houdini's VM can access the physical host system. This test simulates an escape attempt by trying to write to a file path that would typically point to a shared folder if one were configured.

Listing 3.5: Escape Trick: Simulated Write to Host-Mounted Path

```
1 #!/bin/sh
2 # Attempt to write to a path that would be host-mounted if shared
    folders were enabled
3 TOUCH_PATH="/mnt/host_root/hacked.txt"
4
5 echo "Attempting to write to possible host mount at $TOUCH_PATH"
6 echo "HACKED" > "$TOUCH_PATH" 2>/dev/null
```

```
7
8  # Try to detect if the write succeeded
9  if [ -f "$TOUCH_PATH" ]; then
10     echo "Trick passed: Possible host write detected!"
11 else
12     echo "Trick failed: No host write access."
13 fi
```

3.2.3 Execution Inside the VM

The container was executed inside a QEMU-based Buildroot VM that does not expose any host directories and uses no shared folders. The container was started using the following command:

Listing 3.6: Escape Trick: Simulated Write to Host-Mounted Path

```
1 docker run --rm alpine /host_escape_trick.sh
```

Output:

Listing 3.7: Escape Trick: Simulated Write to Host-Mounted Path

```
1 Attempting to write to possible host mount at /mnt/host_root/hacked.
     txt
2 Trick failed: No host write access.
```

No file named `hacked.txt` appeared on the physical host, confirming that the simulated attack was fully contained within the VM.

3.2.4 Implications of Shared Folders

While shared folders are a common method for transferring files between a VM and its physical host, their use inherently weakens the isolation boundary. If a container within the VM has access to a host-mounted directory, it may write or overwrite files

that become visible outside the VM. This creates a deliberate escape route that must be managed with caution.

However, Houdini avoids this problem entirely by not relying on shared folders for interprocess communication or result reporting of tricks. This is because Houdini uses a lightweight Flask-based HTTP server running on the host. When a trick completes execution inside the VM, it sends its result, such as pass/fail status or diagnostic output back to the host via an HTTP POST request over a user-mode QEMU network port.

3.2.5 Baseline Comparison: Without VM Isolation

To better understand the value of Houdini's VM-based architecture, it is useful to compare it to traditional container deployments that run directly on the physical host. In such environments, containers share the host's kernel and rely on namespace and cgroup isolation to enforce security boundaries. While these mechanisms are generally effective, they are not foolproof, especially when advanced Docker features or misconfigurations come into play.

Flags like `--pid=host`, `--net=host`, and `--ipc=host` allow containers to join the host's namespaces. When used, these flags remove key layers of isolation by allowing the container to see or interact with host-level resources such as process IDs, network interfaces, or shared memory segments. These options are sometimes enabled for debugging, performance, or compatibility reasons but significantly expand the container's visibility and control over the host environment.

Even more critically, the use of volume mounts (e.g., `-v /:/host`) can give a container direct access to the host filesystem. If write access is granted intentionally or through misconfiguration, an attacker could overwrite sensitive files such as `/etc/passwd`, replace binaries, or inject persistent malware. In practice, many container escapes and privilege escalations have stemmed from overly permissive volume

mounts or the misuse of the `--privileged` flag, which grants broad access to kernel interfaces and devices.

In contrast, Houdini enforces a strict boundary between the containerized workload and the physical host by placing containers inside a disposable VM. Even if a container is launched with aggressive options like `--privileged` or `--pid=host`, these privileges are scoped only to the VM, not the physical host. This means that even misconfigured or malicious containers cannot directly interact with the physical hosts kernel, devices, or filesystem.

Additionally, Houdini's architecture ensures that test environments are ephemeral. Each container runs inside a fresh VM instance with no persistence beyond the test lifecycle. This further reduces risk by eliminating opportunities for long-term compromise or lateral movement.

Ultimately, this baseline comparison shows that Houdini provides a stronger isolation guarantee than conventional Docker deployments. By decoupling the guest environment from the host through virtualization, Houdini limits the radius of potential container exploits and enforces a security boundary that is significantly harder to breach.

3.2.6 Consideration of VM Escape Attacks

VMs are widely regarded as strong isolation boundaries between guest and host systems. However, they are not immune to attack. A VM escape occurs when code executing inside the guest environment breaks out of its confinement and gains unauthorized access to the host. These attacks typically target vulnerabilities in the hypervisor, emulated device interfaces, or low-level system components.

Several high-profile vulnerabilities demonstrate the feasibility of VM escapes. For example, CVE-2015-3456 (VENOM) exploited a flaw in QEMU's virtual floppy disk controller to achieve arbitrary code execution on the host [1]. Similarly, speculative

execution vulnerabilities such as CVE-2017-5715 (Spectre) [2] and CVE-2019-11135 (ZombieLoad) [3] enabled attackers to leak host memory data from within a guest. These examples, though rare and usually mitigated through timely patches, underscore the importance of minimizing the attack surface of the virtualization layer.

Houdini is not designed to test the security of QEMU or the underlying hypervisor. However, its architecture incorporates multiple safeguards that reduce the risk and impact of VM escape attempts. First, Houdini uses QEMU in user-mode without access to host hardware, kernel modules, or direct device passthrough. Second, the guest OS is a minimal Buildroot-based system, which avoids unnecessary services, drivers, and user-facing software that could be exploited from within the container. Third, common high-risk configurations such as shared folders, USB passthrough, or dynamic device mounting are explicitly avoided.

While no virtualized system is immune to escape-class vulnerabilities, Houdini's design adheres to best practices for secure VM isolation. Its architecture limits exposure to known escape vectors, confines all container execution to a disposable environment, and avoids risky host-guest integration mechanisms. These decisions ensure that even in the presence of an unknown or unpatched vulnerability, the practical risk to the physical host remains minimal.

This design has several advantages:

- It eliminates the need to mount any part of the host filesystem inside the VM.
- It decouples file I/O from test reporting, reducing the attack surface.
- It enables fine-grained control over what data leaves the VM and how it is structured.

3.2.7 Scope and Limitations.

This evaluation focuses on validating Houdini's host isolation guarantees through simulated behavior rather than active exploitation. By design, Houdini does not expose any part of the physical host to the VM, making true host access attempts infeasible without deliberately weakening the environment. As such, the trick used here serves as a canary. It simulates a write to a host-mounted path that would only succeed if isolation were broken through misconfiguration or unexpected leakage. While this approach cannot detect low-level hypervisor vulnerabilities or advanced VM escape techniques, it effectively confirms that Houdini's default configuration does not expose host resources through typical container mechanisms. Stronger adversarial testing, such as injecting known hypervisor exploits or performing fuzzing against QEMU interfaces, is beyond the scope of this evaluation but would complement this work in future studies.

3.2.8 Conclusion

This experiment confirms that Houdini provides strong host isolation by default. Malicious trick code that attempts to write to host-accessible paths is safely confined within the VM. The inclusion of a clearly defined threat model ensures that the assumptions and limits of containment are well-scoped. Furthermore, the use of a Flask-based communication channel, rather than shared folders, enhances isolation by avoiding any direct file-based interaction between the guest and host. Overall, Houdini's VM-based architecture combined with controlled, network-based result reporting offers a robust and secure foundation for container security testing.

3.3 Extensibility Evaluation

Here, we evaluate Houdini's extensibility by introducing a new custom trick to demonstrate that no modifications to the core orchestration logic is required. The goal is to confirm that new security scenarios can be added through configuration alone, and with the goal of highlighting Houdini's modular and scalable testing capabilities.

To implement the new trick, two components were created: (1) a configuration file defining the test matrix, and (2) a trick script that contains the logic executed inside the container.

1. Configuration File

A new configuration file was placed in the `trick_configs/` directory. This file specifies the test parameter space, such as Docker runtime flags and container-level security settings. The following variables were defined:

Listing 3.8: Trick Configuration Parameters

```
pid_mode = [None, "--pid=host"]
cap_kill = [False, True]
cap_sys_ptrace = [False, True]
privileged_opts = [False, True]
apparmor_profiles = ["apparmor=unconfined", "apparmor=docker-default
    "]
user = ["huzi"]
```

Each variable represents a dimension in the test matrix:

- **pid_mode**: Whether to isolate or share the PID namespace with the host.
- **cap_kill**: Whether to include the `CAP_KILL` capability.
- **cap_sys_ptrace**: Whether to include the `CAP_SYS_PTRACE` capability.

- **privileged_opts**: Whether the container runs in `--privileged` mode.
- **apparmor_profiles**: Specifies the AppArmor profile to use.
- **user**: Defines the user identity inside the container.

2. Trick File

In addition to the configuration, a corresponding trick script must be placed in the `tricks/` directory inside the overlay filesystem. This file contains the actual logic to be executed inside the container. For example, a script might attempt to send a signal to a host process or inspect the contents of a protected namespace. In our evaluation, we created a simple trick that checks if a file exists in a container. The script is named to match the configuration and is automatically included in the container image during build time.

Listing 3.9: Trick Configuration Parameters

```
#!/bin/sh

TARGET_FILE="/run/trigger_file"

echo "Checking for existence of $TARGET_FILE"

if [ -f "$TARGET_FILE" ]; then
    echo "Trick passed: File exists."
else
    echo "Trick failed: File not found."
fi
```

By separating trick logic from orchestration, Houdini allows new security tests to be created without modifying the framework itself. As long as a valid configuration file and corresponding trick script are provided, Houdini's pipeline takes care of the

rest.

Execution Flow

Once the new configuration and trick script are in place, the user activates the trick by setting the `TRICK` variable in `constants/houdini_config.py` (e.g., `TRICK = "file_exists"`).

Listing 3.10: Trick Configuration Parameters

```
1 TRICK = "FileExists"
```

Generalizability

The success of this new trick demonstrates more than just a single functional test, it confirms the correctness and generality of Houdini's extensibility model. This is because Houdini treats all tricks as interchangeable components, executed through the same standardized interface. Once a new trick is defined and referenced through a configuration file, Houdini automatically performs the following actions regardless of the trick's internal logic:

- Loads the associated configuration without requiring logic changes.
- Injects the corresponding trick script into the container image during build.
- Executes the container with the correct flags inside an isolated VM.
- Captures trick output through a uniform logging mechanism.

There is no tight coupling between the orchestration logic and the contents of the trick. The system does not assume or validate the purpose of the script, it simply executes it. Therefore, if one trick can be successfully added, executed, and logged using this path, any trick that follows the same structure can be integrated as well.

This property demonstrates that Houdini's design is not tailored to specific tests but is built around a flexible, configuration-driven framework. As long as future tricks conform to the same simple interface, they can be added without modifying the system, validating that the extensibility mechanism is generalizable.

Conclusion

This workflow demonstrates Houdini's modular architecture. Test logic (the trick file) and configuration are fully decoupled from orchestration logic. New scenarios can be added with minimal effort, requiring only a trick script and a config file. The core system remains unchanged, which makes Houdini highly extensible and suitable for evaluating new runtime behaviors, emerging vulnerabilities, or evolving isolation mechanisms in a scalable and automated fashion.

3.4 Version Variability Support

Container runtime behavior evolves significantly over time. Docker, runc, and the Linux kernel introduce new APIs, deprecate flags, and change default security configurations across versions. To remain effective, a security test framework like Houdini must support this variability without requiring manual adaptation. This evaluation asks: Can Houdini reliably build and execute security tests across historical and modern container ecosystems with no changes to its internal logic?

To answer this, we constructed a version matrix spanning over 70 Docker releases and their associated runc versions, grouped into three eras: early (e.g., 17.03), mid-range (e.g., 20.10), and modern (e.g., 27.0.3). Each combination was tested using a consistent trick designed to detect CVE-2024-21626 (see Case Study 1).

Importantly, Houdini required no code or script modifications throughout this evaluation. Test logic remained unchanged. The orchestration layer, trick execution

pipeline, and result reporting functioned identically across all tested Docker and runc versions. These findings validate Houdini's suitability as a version-agnostic security regression testing framework, capable of catching both intended fixes and unintended regressions across evolving container platforms.

This level of compatibility makes Houdini suitable not just for point-in-time security evaluation, but also for long-term regression tracking, archival vulnerability research, and continuous integration pipelines that must accommodate diverse and evolving container environments.

Table 3.4: Docker and runc Versions Validated with Houdini

Docker Version	**runc Version**	**Kernel Version**	**Validated**
17.03.0-ce	1.0.0-rc2	6.0	✓
17.03.1-ce	1.0.0-rc2	6.0	✓
17.03.2-ce	1.0.0-rc2	6.0	✓
17.04.0-ce	1.0.0-rc2	6.0	✓
17.05.0-ce	1.0.0-rc2	6.0	✓
17.06.0-ce	1.0.0-rc3	6.0	✓
17.06.1-ce	1.0.0-rc3	6.0	✓
17.06.2-ce	1.0.0-rc3	6.0	✓
17.09.0-ce	1.0.0-rc4+dev	6.0	✓
17.09.1-ce	1.0.0-rc4+dev	6.0	✓
17.12.0-ce	1.0.0-rc4+dev	6.0	✓
17.12.1-ce	1.0.0-rc4+dev	6.0	✓
18.03.0-ce	1.0.0-rc5	6.0	✓
18.03.1-ce	1.0.0-rc5	6.0	✓
18.06.0-ce	1.0.0-rc5+dev	6.0	✓
18.06.1-ce	1.0.0-rc5+dev	6.0	✓
18.06.2-ce	1.0.0-rc5+dev	6.0	✓
18.06.3-ce	1.0.0-rc5+dev	6.0	✓
18.09.0	1.0.0-rc5+dev	6.0	✓

Continued on next page

Table 3.4 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
18.09.1	1.0.0-rc6+dev	6.0	✓
18.09.2	1.0.0-rc6+dev	6.0	✓
18.09.3	1.0.0-rc6+dev	6.0	✓
18.09.4	1.0.0-rc6+dev	6.0	✓
18.09.5	1.0.0-rc6+dev	6.0	✓
18.09.6	1.0.0-rc6+dev	6.0	✓
18.09.7	1.0.0-rc8	6.0	✓
18.09.8	1.0.0-rc8	6.0	✓
18.09.9	1.0.0-rc8	6.0	✓
19.03.0	1.0.0-rc8	6.0	✓
19.03.1	1.0.0-rc8	6.0	✓
19.03.10	1.0.0-rc10	6.0	✓
19.03.11	1.0.0-rc10	6.0	✓
19.03.12	1.0.0-rc91	6.0	✓
19.03.13	1.0.0-rc10	6.0	✓
19.03.14	1.0.0-rc10	6.0	✓
19.03.15	1.0.0-rc10	6.0	✓
20.10.0	1.0.0-rc92	6.0	✓
20.10.10	1.0.2	6.0	✓
20.10.11	1.0.2	6.0	✓
20.10.12	1.0.2	6.0	✓
20.10.13	1.0.3	6.0	✓
20.10.14	1.0.3	6.0	✓
20.10.15	1.1.1	6.0	✓
20.10.16	1.1.1	6.0	✓
20.10.17	1.1.2	6.0	✓
20.10.18	1.1.4	6.0	✓
20.10.19	1.1.4	6.0	✓
20.10.20	1.1.4	6.0	✓
20.10.21	1.1.4	6.0	✓

Continued on next page

Table 3.4 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
20.10.22	1.1.4	6.0	✓
20.10.23	1.1.4	6.0	✓
20.10.24	1.1.5	6.0	✓
23.0.0	1.1.4	6.0	✓
23.0.1	1.1.4	6.0	✓
23.0.2	1.1.4	6.0	✓
23.0.3	1.1.5	6.0	✓
23.0.4	1.1.5	6.0	✓
23.0.5	1.1.5	6.0	✓
23.0.6	1.1.7	6.0	✓
24.0.0	1.1.7	6.0	✓
24.0.1	1.1.7	6.0	✓
24.0.2	1.1.7	6.0	✓
24.0.3	1.1.7	6.0	✓
24.0.4	1.1.7	6.0	✓
24.0.5	1.1.8	6.0	✓
24.0.6	1.1.9	6.0	✓
24.0.7	1.1.9	6.0	✓
24.0.8	1.1.11	6.0	✓
24.0.9	1.1.12	6.0	✓
25.0.0	1.1.11	6.0	✓
25.0.1	1.1.11	6.0	✓
25.0.2	1.1.12	6.0	✓
25.0.3	1.1.12	6.0	✓
25.0.4	1.1.12	6.0	✓
25.0.5	1.1.12	6.0	✓
26.0.0	1.1.12	6.0	✓
26.0.1	1.1.12	6.0	✓
26.0.2	1.1.12	6.0	✓
26.1.0	1.1.12	6.0	✓

Continued on next page

Table 3.4 – *continued from previous page*

Docker Version	runc Version	Kernel Version	Validated
26.1.1	1.1.12	6.0	✓
26.1.2	1.1.12	6.0	✓
26.1.3	1.1.12	6.0	✓
26.1.4	1.1.12	6.0	✓
27.0.1	1.1.13	6.0	✓
27.0.2	1.1.13	6.0	✓
27.0.3	1.1.13	6.0	✓

Conclusion

In total, Houdini validated 73 Docker versions paired with 20 distinct runc releases, all running on a consistent Linux 6.0 kernel. Every combination succeeded without requiring trick-specific logic changes or framework adaptation. This breadth affirms Houdini's resilience to version-induced divergence and runtime instability across over a decade of Docker ecosystem evolution.

3.5 Performance Benchmarking

Understanding Time Metrics in Microbenchmarking

In performance benchmarking, especially in systems research and software profiling, several time-based metrics are commonly used to assess how a program or process utilizes computing resources. These include wall clock time, user time, system time, and CPU utilization. Each of these captures a different aspect of execution behavior and provides insight into how efficiently a task runs.

Wall clock time, also known as elapsed time or real time, is the total time taken from the start of a program to its completion. It includes all delays, such as waiting for I/O operations, context switches, or sleep intervals. Wall clock time is measured

as the actual duration perceived by a user, typically using timestamps recorded before and after the process runs.

User time refers to the amount of CPU time spent running code in user space. This includes the program's own instructions and any library code it uses. It excludes time spent in the kernel or operating system. This metric is measured by the operating system and tracks how much CPU time was directly consumed by the application itself.

System time (or kernel time) accounts for the time spent by the CPU in the operating system kernel on behalf of the program. This includes handling system calls, managing I/O, allocating memory, and other interactions with hardware or privileged OS routines. Like user time, it is reported by the operating system's process accounting.

CPU utilization is the percentage of total available CPU time that the process actively used during its lifetime. It is computed as the ratio of the sum of user and system time to the wall clock time. High CPU utilization indicates that the process is CPU-bound, meaning it spends most of its time doing computations. Low CPU utilization may suggest the process is I/O-bound or waiting on external events.

Together, these metrics offer a multifaceted view of a program's execution profile. They help developers identify bottlenecks, assess performance regressions, and understand whether inefficiencies stem from computation, system overhead, or external delays.

3.5.1 Microbenchmark

Making the Root Filesystem

To assess the performance of Houdini's build and setup pipeline, we independently microbenchmarked the `mkfs` stage, the process responsible for creating the root filesystem used by the VM. This stage includes both formatting the disk image and pop-

ulating it with essential binaries, configuration files, and trick scripts drawn from Houdini's overlay directory.

Using the `/usr/bin/time` utility, we measured a typical execution time of `60.21 seconds`. The breakdown revealed a **user time** of 19.43 seconds and a **system time** of 45.97 seconds, resulting in an overall CPU utilization of approximately **63%**. These figures suggest that while the operation involves meaningful computation, a significant portion of the work is I/O-bound—largely due to file copying, directory creation, and image formatting tasks.

Table 3.5: Performance Metrics for `mkfs` Stage

Metric	**Value**
Wall-clock time	60.21 seconds
User time	19.43 seconds
System time	45.97 seconds
CPU utilization	63%

It is important to mention that the `mkfs` stage is a one-time setup cost per session. Once the filesystem image is created, it can be reused across multiple Houdini runs without needing to re-execute the build step. This reuse dramatically improves performance in iterative workflows. In other words, By avoiding redundant execution of mkfs, subsequent Houdini runs skip over this 60-second cost entirely. In large batch runs or iterative experiments, this can yield cumulative time savings of several minutes or more. However, any modification to the overlay, such as adding or updating a trick requires rerunning `mkfs` to regenerate the root filesystem and incorporate the changes.

Booting the Virtual Machine

To assess the overhead of launching Houdini's virtual machine environment, we conducted a microbenchmark of the VM startup process. The VM is started using QEMU with a prebuilt Buildroot root filesystem and a custom Linux kernel. The system is

configured to launch a shell and immediately power off after a fixed delay, ensuring consistency in measurement.

We measured the total wall-clock time required to boot the VM. The benchmark was performed using the `/usr/bin/time -v` utility, capturing both user and system time. The wall-clock time for this process was approximately 28.47 seconds. The user time recorded was 3.52 seconds, while the system time was 9.38 seconds. Overall, the job utilized roughly 45% of the CPU during execution.

Table 3.6: Performance Metrics for VM Boot

Metric	**Value**
Wall-clock time	28.47 seconds
User time	3.52 seconds
System time	9.38 seconds
CPU utilization	45%

These results indicate that the boot process introduces a moderate overhead in Houdini's workflow, primarily due to QEMU's emulation and the initialization of the Linux kernel and user space environment. Despite this, the startup time remains stable across multiple invocations, providing a consistent and reliable baseline for empirical testing. This microbenchmark reinforces the practicality of using QEMU as a platform for repeatable, isolated container security experiments.

Building the Docker Image

To evaluate the performance overhead of Docker image creation within the Houdini pipeline, we conducted a microbenchmark of the `docker build` process. The benchmark was executed using the `/usr/bin/time -v` utility to capture detailed runtime metrics, including user time, system time, wall-clock time, and CPU utilization.

The wall-clock time to complete the initial build process was measured at approximately `1 minute and 13.04 seconds`. During this time, the process consumed `0.33 seconds` of user CPU time and `0.30 seconds` of system CPU time. The CPU uti-

lization was reported at `0%`, indicating that the build process was largely I/O-bound, likely due to reading files, writing intermediate layers, and performing setup tasks for the container image.

Table 3.7: Performance Metrics for Docker Image Build (Cold Start)

Metric	**Value**
Wall-clock time	1 minute 13.04 seconds
User time	0.33 seconds
System time	0.30 seconds
CPU utilization	0%

It is important to note that this benchmark reflects the case where the Docker image did not previously exist. The initial build incurs higher latency due to downloading base layers, compiling dependencies, and writing the final image to disk. In subsequent builds where layers are cached and the image already exists locally, the build time is significantly reduced, and CPU utilization may increase due to reduced I/O waits.

These results highlight that while the initial Docker build may introduce measurable delay, especially in cold-start scenarios, this cost can be mitigated through caching and reuse of previously built images. For workflows like Houdini, where consistency and speed are critical, avoiding redundant rebuilds is key to maintaining performance.

Running a Trick in Docker

To evaluate the runtime characteristics of executing a Houdini trick, we measured the performance of the command `docker run -it 504de41abf9b`. The benchmark revealed minimal resource usage, with `0.10 seconds` of user time and `0.07 seconds` of system time. The overall CPU utilization was measured at `9%`, suggesting that the trick involved limited computation and was mostly I/O-bound.

These results highlight Houdini's suitability for high-throughput, low-latency em-

Table 3.8: Your Caption

Metric	Value
User time	0.10 seconds
System time	0.07 seconds
CPU utilization	9%

pirical testing, with minimal impact on system resources per trick execution.

Baseline Comparison Outside Houdini

To contextualize Houdini's efficiency, we measured a baseline workflow executed manually, without the automation Houdini provides. This baseline consisted of starting the VM, building the Docker image, and executing a security trick, which were all performed via shell scripts without caching or orchestration.

We used `/usr/bin/time -v` to capture system-level performance metrics. The manually executed baseline took approximately 55.70 seconds of wall-clock time, with a user time of 22.51 seconds and a system time of 18.12 seconds. The CPU utilization for this task was 73%, indicating moderate computational activity with some idle time.

Table 3.9: Performance Metrics for Manually Executed Baseline

Metric	Value
Wall-clock time	55.70 seconds
User time	22.51 seconds
System time	18.12 seconds
CPU utilization	73%

Compared to Houdini's streamlined execution, which achieves the same test in a fraction of the time through reuse of cached artifacts and fewer manual dependencies, this demonstrates that Houdini significantly reduces runtime overhead.

3.6 Batch Trick Execution

To evaluate Houdini's performance under batch workloads, we executed a session consisting of 32 distinct tricks across a single version of Docker and kernel. The goal was to measure the system's ability to handle high-throughput empirical testing within a single execution context. This benchmark focused on key metrics such as total runtime, CPU usage, and failure rates.

The benchmark was conducted using the `/usr/bin/time -v` utility to capture detailed performance statistics. The total wall-clock time for executing all 32 tricks was approximately 1 minute and 6.80 seconds. During this period, the system reported a user time of 2.75 seconds and a system time of 0.48 seconds, with an overall CPU utilization of just 4%. These results indicate that while the batch execution is time-bound primarily by I/O and process orchestration overhead, the actual CPU demand remains minimal, suggesting scalability across larger test sets.

Table 3.10: Performance Metrics for Batch Execution of 32 Tricks

Metric	**Value**
Total tricks executed	32
Wall-clock time	66.80 seconds
User time	2.75 seconds
System time	0.48 seconds
CPU utilization	4%

Chapter 4

Case Studies

4.0.1 Case Study 1: Evaluating the accuracy of CVE-2024-21626

CVE-2024-21626 [4] is a container escape vulnerability in runc, the low-level container runtime used by Docker and other container engines. It allows a malicious container to escape its isolation and execute arbitrary commands on the host system.

The root cause of this vulnerability lies in how runc handles cgroups for containers, which are used to manage system resources like CPU and memory. During container creation, runc opens a file descriptor to the host's `/sys/fs/cgroup` directory to configure resource limits. This descriptor remains open throughout the container's lifecycle and is exposed inside the container via `/proc/self/fd/`, which lists all open file descriptors of the running process. The vulnerability arises when an attacker sets the working directory in the Dockerfile to point to one of these exposed descriptors, such as `/proc/self/fd/N`, where `N` refers to the specific file descriptor inherited from runc that points to the host's cgroup hierarchy. This causes the container's current working directory to resolve to a privileged host path. As a result, all subsequent relative file operations in the container are interpreted from the host's cgroup filesystem rather than the container's own root, allowing unintended access or modification of host resources.

This behavior is enabled by Linux's treatment of `/proc/self/fd/N` as dynamic symlinks to open file descriptors. When dereferenced, these symlinks resolve to the original host-side path, regardless of container filesystem restrictions. Because runc and Docker do not restrict access to inherited file descriptors or sanitize the working directory path during initialization, attackers can exploit this oversight to escape isolation. By simply changing the container's working directory to a leaked file descriptor, a malicious container can directly interact with host-managed control groups. To test the exploitability of CVE-2024-21626, we designed a Houdini trick that executes the following steps on Docker and runc versions listed in Table 4.1:

1. Set the working directory to `/proc/self/fd/<fd>`, pointing to a controlled file descriptor.

2. Attempt to break out of the container by leveraging the misconfigured working directory.

3. Validate whether commands can be executed on the host system.

The CVE for this vulnerability indicates that the affected runc versions are $\geq$ `1.0.0-rc93`, $\leq$ `1.1.11`. This suggests the vulnerability was introduced in runc version rc93 and fixed in version 1.1.12. However, our tests conducted with Houdini revealed a significant discrepancy with this assertion. Our testing showed that earlier runc versions, such as `1.0.0-rc1`, `1.0.0-rc2`, `1.0.0-rc3`, and `1.0.0-rc4` also exhibited the same vulnerability. This discovery expands the scope of the CVE by revealing that the issue existed well before version `rc93`.

This discrepancy highlights a recurring issue in vulnerability disclosure processes: CVEs often rely on assumptions from static code analysis or limited testing. Without rigorous empirical validation across historical versions, critical edge cases can be missed. In the case of CVE-2024-21626, this oversight could have allowed vulnerable

deployments relying on older runc versions to remain unpatched under the false belief that they were unaffected.

4.0.2 Case Study 2

In this evaluation, we examine whether a Docker container running as a non-privileged user with `UID 1000` can interact with and manipulate host processes that are also owned by `UID 1000`. Under a default Docker configuration, the PID namespace provides process isolation by restricting a container's visibility and access to processes outside its own namespace. However, when the container is launched with the `--pid=host` flag, it shares the host's PID namespace, allowing it to enumerate and interact with host processes. This configuration effectively bypasses one of the core isolation mechanisms provided by containers. To assess the security implications of this namespace sharing, we investigate whether process-level interactions, such as sending signals or attaching debuggers are permitted between the container and host processes when UID ownership is aligned. The testing environment for this case study is in Table 4.2.

Case study 2's results are summarized in Table **??**, where all pairs of flags were tested using Houdini, and where the cells show what functionality was possible when launching containers with the pair of flags intersecting the rows and columns of Table **??**.

Our objective here is to investigate how various security flags interact with each other, and to determine the minimum set of privileges required for a container to manipulate host processes. Specifically, we looked at the following flags:

- `--pid=host`
- `CAP_KILL`
- `CAP_SYS_PTRACE`

- `--privileged`
- `apparmor=docker-default`

The evaluation began by assessing whether a container launched with `--pid=host` could list host processes using `ps aux`. As expected, the container displayed all processes running on the host, confirming that sharing the host PID namespace fully exposes the host's process table to the container.

Following this, we tested whether a container process could send a `SIGKILL` signal to a host process owned by `UID 1000`. The container was launched with `--pid=host`, allowing it to see and interact with host processes. Despite not being granted `CAP_KILL`, the signal was successfully delivered. This is because by default, both processes share the same UID and PID namespace, and under standard Linux permission checks, a process can send signals to another process it owns by UID without requiring special capabilities. However, if the target host process is owned by a different UID (another non-root user), `CAP_KILL` is required for the signal to be delivered. That capability must also be effective in the user namespace of the target process. Similarly, signaling host processes owned by `UID 0` remains restricted even with `CAP_KILL`, unless the container process itself is also running as root.

Combining `--pid=host` with `CAP_SYS_PTRACE` enables nearly the same level of interaction with host processes as granting `--privileged` mode. Specifically, it allows a containerized process to list host processes, send signals, and attach a debugger to them, all key capabilities typically associated with privileged containers. However, unlike `--privileged`, this more targeted configuration avoids giving the container unnecessary capabilities such as access to all of the host's devices. These results demonstrate that it is unnecessary to over-privilege containers when finer-grained controls can achieve the same functional outcomes with a significantly better security posture.

When the container is confined by AppArmor's `docker-default` profile, its ability to interact with host processes becomes significantly restricted. Specifically, `docker-default` explicitly denies system calls such as `signal` and `ptrace`, which are required for sending signals and attaching debuggers to other processes. As a result, even if the container shares the PID namespace with the host and possesses the necessary capabilities, these operations will fail.

However, including the `--privileged` flag when starting the container overrides AppArmor confinement by disabling the `docker-default` profile entirely. This lifts all AppArmor-imposed restrictions, allowing the container full access to the previously restricted system calls. As a result, operations that were previously blocked, such as sending signals or attaching debuggers to host processes are now permitted, even if AppArmor would normally deny them.

It is also important to note that granting capabilities such as `CAP_KILL` and `CAP_SYS_PTRACE` alone is not sufficient for interacting with host processes unless the container shares the host's PID namespace. Without the `--pid=host` flag, the container operates in its own isolated PID namespace, meaning it cannot see or reference host PIDs at all. As a result, any attempt to send signals or attach to host processes will fail.

We note that the explanation above was derived after substantial amount of testing and analysis. The Docker documentation alone does not clarify this behavior. Instead, users are expected to understand both the Docker tooling and the underlying Linux kernel security model to correctly select these flags.

When the container shares the same user namespace as the host (or has aligned UID mappings), and the container process runs under the same UID as the host processes, it can bypass security restrictions that would normally prevent such interactions. In this case, the container may be able to perform operations like sending signals or attaching debuggers to host processes, even without elevated capabilities

like `CAP_KILL` or `CAP_SYS_PTRACE`. These capabilities, which are typically required for such actions, are not necessary when the container and host share UID mappings, as the container process is considered the same user as the host processes it targets.

This case study highlights a core strength of Houdini, which is its ability to systematically explore and identify the least privilege for a given operation. By allowing users to declaratively modify container configurations, Houdini makes it easy to test a wide range of privilege combinations in a repeatable and automated way. Instead of relying on trial and error or defaulting to the overly permissive `--privileged` mode, users can incrementally grant permissions and observe the effect of each change. Houdini thus enables security practitioners, developers, and researchers to precisely determine the minimum set of permissions required for a given trick to succeed. As a result, Houdini not only serves as a testing framework for container escapes and misconfigurations, but also as a practical tool for privilege minimization and hardening container deployments.

This case study highlights the complexity of security policies in containerized environments, highlighting that the runtime behavior of security configurations is not always straightforward or predictable. While administrators may configure a container with `--pid=host` to enable the container to monitor host processes, there is an assumption that certain operations, such as sending signals or attaching debuggers remain restricted. However, these assumptions may not hold at runtime due to the interplay of various factors such as user namespaces and UID mappings.

4.0.3 Case Study 3 - Evaluating Docker Privileged Port Forwarding Restrictions

In this case study, we evaluate Docker's network isolation mechanisms to determine how well they enforce privileged port binding restrictions. Starting from Docker version 20.03, unprivileged ports start at 0 instead of 1024, effectively removing the

need for root to bind to privileged ports. Table 4.5 presents the results of this change.

This case study highlights how discrepancies between Docker versions in handling privileged port binding can create gaps between administrators' intended confinement policies and actual enforcement. A change in Docker's handling of binding to privileged ports, such as the removal of the NET_BIND_SERVICE requirement starting from Docker 20.03, may allow containers to bypass restrictions even when policies are set to prevent it.

For example, administrators might configure a container to be restricted from binding to privileged ports, but if Docker is running a version where the NET_BIND_SERVICE capability is no longer required for root containers (like in Docker 20.03 and beyond), the container could still bind to these ports despite the policy specifying otherwise. This creates a mismatch between the intended policy (restricting privileged port binding) and the actual enforcement (where containers can still bind to those ports in certain Docker versions).

Once again, Houdini helps identify these gaps by systematically testing container configurations across different Docker versions. By running different Docker versions and security policies, Houdini verifies whether containers are correctly enforcing the intended restrictions on privileged port binding. For instance, Houdini could test whether containers configured to prevent binding to privileged ports are still able to bind to those ports in Docker versions 20.03 or later, where the security model has changed. If a container running with Docker 20.03 or newer can bind to privileged ports despite the configuration specifying otherwise, Houdini will detect this discrepancy and highlight the gap between the intended policy and actual enforcement.

This case study also demonstrates a clear example of configuration drift arising not from explicit changes made by administrators, but from underlying changes in the Docker runtime itself. As Docker evolves, fundamental shifts in its privilege model, such as the redefinition of which ports are considered privileged and the removal of

the NET_BIND_SERVICE requirement for certain operations can silently undermine previously established security policies. Even if an organization maintains the same container configuration and security policies, upgrading Docker to a newer version can inadvertently expand container capabilities, allowing behaviors that were once restricted. This version-induced drift can lead to a false sense of security, where administrators believe containers are still confined by earlier policy expectations, when in fact those policies are no longer fully enforced.

4.0.4 Case Study 4 - Evaluating ptrace Enforcement Across Linux Distributions

To assess the effectiveness of Linux kernel mechanisms in enforcing process isolation within containerized environments, we conducted a series of controlled experiments targeting the ptrace system call under various configurations. These experiments were designed to evaluate how different kernel versions, Yama security module settings, and process capabilities affect the ability of a containerized process to trace other processes, both inside and outside its container boundary.

Table 4.1: runc and crun Versions Against CVE-2024-21626

Docker Version	Runtime Version	Leaked FD	Exploit Success
runc			
-	runc 0.0.9	-	✗
1.12.0	runc 1.0.0-rc1	/proc/self/fd/4	✓
17.03.0-ce	runc 1.0.0-rc2	/proc/self/fd/4	✓
17.06.1-ce	runc 1.0.0-rc3	/proc/self/fd/5	✓
17.09.0-ce	runc 1.0.0-rc4	/proc/self/fd/5	✓
-	runc 1.0.0-rc8	-	✗
-	runc 1.0.0-rc10	-	✗
-	runc 1.0.0-rc91	-	✗
20.10.17	runc 1.1.2	/proc/self/fd/9	✓
23.0.1	runc 1.1.4	/proc/self/fd/8	✓
23.0.5	runc 1.1.5	/proc/self/fd/8	✓
24.0.2	runc 1.1.7	/proc/self/fd/9	✓
24.0.5	runc 1.1.8	/proc/self/fd/9	✓
24.0.7	runc 1.1.9	/proc/self/fd/9	✓
24.0.0	runc 1.1.11	/proc/self/fd/9	✓
-	runc 1.1.12	-	✗
-	runc 1.1.13	-	✗
-	runc 1.1.14	-	✗
crun			
-	crun 0.15	-	✗
-	crun 0.17	-	✗
-	crun 1.0.0	-	✗
-	crun 1.2.0	-	✗
-	crun 1.3.0	-	✗
-	crun 1.5.0	-	✗
-	crun 1.6.0	-	✗
-	crun 1.7.0	-	✗
-	crun 1.8.0	-	✗
-	crun 1.9.0	-	✗
-	crun 1.10.0	-	✗
-	crun 1.11	-	✗
-	crun 1.12	-	✗
-	crun 1.13	-	✗
-	crun 1.14	-	✗
-	crun 1.15	-	✗
-	crun 1.16	-	✗
-	crun 1.17	-	✗
-	crun 1.18	-	✗
-	crun 1.19	-	✗
-	crun 1.20	-	✗
-	crun 1.21	-	✗

Table 4.2: Case Study 2 Testing Environment

Component	**Specification**
Docker Version	27.0.3
Container Runtime	`runc` 1.1.13

Table 4.3: Ability for Containers to Interact with Host Processes Across Yama Settings (Ubuntu Kernel 6.8.12)

Security Configuration	**Yama Scope**	`--pid=host`	`CAP_KILL`	`CAP_SYS_PTRACE`	`--privileged`	`apparmor=docker-default`	`--pid=host + apparmor=docker-default`
`--pid=host`	0	♢ △	♢ △	♢ △ ⋆	♢ △ ⋆	♢	♢
	1	♢ △	♢ △	♢ △ ⋆	♢ △ ⋆	♢	♢
	2	♢ △	♢ △	♢ △	♢ △	♢	♢
	3	♢ △	♢ △	♢ △	♢ △	♢	♢
`CAP_KILL`	0	♢ △					♢
	1	♢ △					♢
	2	♢ △					♢
	3	♢ △					♢
`CAP_SYS_PTRACE`	0	♢ △ ⋆					♢
	1	♢ △ ⋆					♢
	2	♢ △					♢
	3	♢ △					♢
`--privileged`	0	♢ △ ⋆					♢ △ ⋆
	1	♢ △ ⋆					♢ △ ⋆
	2	♢ △					♢ △
	3	♢ △					♢ △
`apparmor=docker-default`	0	♢					♢
	1	♢					♢
	2	♢					♢
	3	♢					♢
`--pid=host + apparmor=docker-default`	0	♢	♢	♢	♢ △ ⋆	♢	♢
	1	♢	♢	♢	♢ △ ⋆	♢	♢
	2	♢	♢	♢	♢ △	♢	♢
	3	♢	♢	♢	♢ △	♢	♢

Table 4.4: *

Tests run as a non-root user (UID=1000) against a host process owned by UID 1000.
♢ List Host Process △ Send Signal to Host Process ⋆ Attach Debugger to Host Process

Table 4.5: Ability for a container to bind to privileged ports

`Docker Version`	`NET_BIND_SERVICE + root`	`NET_BIND_SERVICE + non-root user`	`root`	`non-root user`	`Default Containe`
19.03.9	✓	✗	✗	✗	✗
19.03.0	✓	✗	✗	✗	✗
20.10.0	✓	✗	✗	✗	✗
20.10.3	✓	✗	✗	✗	✗
23.0.0	✓	✓	✓	✗	✗
25.0.0	✓	✓	✓	✗	✗
27.0.1	✓	✓	✓	✗	✗

Table 4.6: *

Note: A non-root user implies a user account that does not have administrative privileges, (not root-less Docker).

Table 4.7: ptrace Behavior Across Distributions with Default Yama Scopes

Distribution	**Kernel Version**	**Yama Scope (Default)**	**CAP_SYS_PTRACE**	**Result**
Debian 12	6.1 (Bookworm)	1	Yes	Success
Debian 12	6.1 (Bookworm)	1	No	Fail
Debian 8	3.16 (Jessie)	0	Yes	Success
Debian 8	3.16 (Jessie)	0	No	Success
Fedora 40	6.7	2	Yes	Fail
Fedora 40	6.7	2	No	Fail
Ubuntu 24.04	6.8.12	1	Yes	Success
Ubuntu 24.04	6.8.12	1	No	Fail
Ubuntu 22.04	5.15 (LTS)	1	Yes	Success
Ubuntu 22.04	5.15 (LTS)	1	No	Fail
Ubuntu 16.04	4.4 (LTS)	0	Yes	Success
Ubuntu 16.04	4.4 (LTS)	0	No	Success
Alpine 3.19	6.1	0	Yes	Success
Alpine 3.19	6.1	0	No	Success
Arch Linux	6.8 (Rolling)	1	Yes	Success
Arch Linux	6.8 (Rolling)	1	No	Fail
CentOS 7	3.10	1	Yes	Success
CentOS 7	3.10	1	No	Fail
CentOS 6	2.6.32	0	Yes	Success
CentOS 6	2.6.32	0	No	Success

Bibliography

[1] CrowdStrike. VENOM: Virtualized Environment Neglected Operations Manipulation. Online, 2015. https://www.crowdstrike.com/blog/cve-2015-3456-venom/.

[2] Paul Kocher, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 1–19, 2019.

[3] Michael Schwarz, Daniel Gruss, Moritz Lipp, and Stefan Mangard. ZombieLoad Attack. Online, 2019. https://zombieloadattack.com/.

[4] Open Containers Maintainers. runc: File descriptor leak leading to container breakout (CVE-2024-21626). https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv, 2024. Accessed: 2024-04-28.